

📱 Pregnancy App - Developer Handbook

Version: 1.2.0

Last Updated: 2025-11-28

Tech Stack: React Native (Expo), TypeScript, Firebase (Auth, Firestore), i18next

1. 📖 Introduction

This project is a comprehensive pregnancy tracking application designed to support expectant mothers from week 1 to week 40. It features weekly development updates, health reminders, medical appointment tracking, a chatbot assistant, and a rich library of articles and supplements.

Key Features

- **Weekly Tracking:** Baby size, development updates, mom's body changes.
- **Health Reminders (V1.2):** Customizable daily reminders (hydration, meds, etc.) with local notifications.
- **Baby Messages (V1.2):** Daily emotional/fun messages from the baby.
- **Calendar:** Medical and personal appointment management.
- **Resources:** Curated articles and supplement safety guides.
- **Chatbot:** AI-powered assistant with "Red Flag" detection for medical emergencies.
- **Multi-language:** Built for FR, EN, AR (currently content is FR only).

2. ■ Getting Started

Prerequisites

- Node.js (v18+)
- npm or yarn
- Expo CLI (npm install -g expo-cli)
- iOS Simulator (Mac) or Android Emulator

Installation

1.

Clone the repository:

```
git clone <repo-url>
```

```
cd pregnancy-app
```

2.

Install dependencies:

`npm install`

3.

Environment Setup: Create a

`.env`

file in the root directory with your Firebase credentials:

`EXPO_PUBLIC_FIREBASE_API_KEY=...`

`EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=...`

`EXPO_PUBLIC_FIREBASE_PROJECT_ID=...`

EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=...

EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=...

EXPO_PUBLIC_FIREBASE_APP_ID=...

4.

Run the App:

npm start

Press 'i' for iOS, 'a' for Android

3. 📁 Architecture & Structure

The project follows a feature-based and service-oriented architecture.

Directory Structure (app/src)

src/

📁 components/ # Reusable UI components

📁 common/ # Generic (Card, Button, Tag...)

📁 calendar/ # Calendar-specific (MonthView, EventList...)

📁 home/ # Home-specific (BabyFactsCard, WeekReminders...)

📁 config/ # Configuration files (firebase.ts)

context/ # React Contexts (Global State)

i18n/ # Internationalization (locales, config)

screens/ # Screen components (Pages)

Auth/ # Login, Register, Language

Core/ # Main Tabs (Home, Calendar, Resources...)

Details/ # Detail views (Article, Supplement...)

Onboarding/ # Initial setup

services/ # Business logic & API calls

theme/ # Design system (colors, typography)

types/ # TypeScript interfaces (Single source of truth)

utils/ # Helper functions (dates, parsing)

Navigation (App.tsx)

We use react-navigation with a mix of Stacks and Tabs:

1. **AuthStack**: LanguageSelect → AuthChoice → Login / Register.

2. **MainTabs**

(Bottom Tab Bar):

- **Home**: HomeStack (Home → Details → Settings)
- **Calendar**: CalendarStack
- **Rappels**: RemindersStack (New V1.2)
- **Ressources**: ResourcesStack
- **Chat**: ChatbotScreen

3. **OnboardingStack**: For new users to set up profile.

State Management

- **AuthContext**: Manages Firebase User state (user, loading, login, logout).
- **PregnancyContext**
: Manages User Profile & Pregnancy Calculation (profile

```
,
pregnancyInfo
,
setProfile
).
```

- **Key Logic:** Calculates current week based on LMP (Last Menstrual Period).

4. 📖 Features Deep Dive

4.1 Home Screen (HomeScreen.tsx)

The central hub. It displays:

- **Header:** Greeting, Week/Day navigation.
- **V3 Cards:** BabyFactsCard, MomTipsCard (Gradient styled, parsed from V3 data).
- **WeekRemindersCard:** Top 2 reminders/tasks for the week.
- **Recommendations:** Horizontal list of Articles/Supplements linked to the current week.

4.2 Reminders System (V1.2) (RemindersScreen.tsx)

A complex feature managing local notifications.

- **Tabs:** "Rappels" (Health) and "Tâches" (To-Do).
- **Persistence**
 - :
 - **Guest:** AsyncStorage (@reminder_settings_v1).
 - **Auth:** Firestore (userReminderSettings collection).

- **Notifications**

: Uses

expo-notifications

.

- Logic: notificationService.ts handles scheduling (CalendarTrigger).
- Intensity: Users choose 3, 5, or 7 times/day.

4.3 Baby Messages (V1.2)

Daily emotional messages from the baby.

- **Data:** 280 unique messages in Firestore babyMessages.
- **Logic:** babyMessageService.ts picks the message for Week X, Day Y.
- **Scheduling:** One notification per day (default 10:00 AM).

4.4 Calendar (CalendarScreen.tsx)

- **Templates:** Pre-filled medical appointments based on pregnancy week (Firestore calendarTemplates).
- **User Events:** Custom appointments added by the user.
- **Views:** Toggle between Month View and List View.

4.5 Chatbot (ChatbotScreen.tsx)

- **Red Flags:** Analyzes user input for keywords (e.g., "bleeding") against redFlags collection.
 - **Safety:** If a red flag is detected, shows an immediate medical warning.
-

5. 📁 Data Layer (Firestore)

Core Collections

- weeks (40 docs): Static content for each week.
- articles (20 docs): Health articles.
- supplements (15 docs): Supplement safety info.
- babyMessages (280 docs): Daily messages.

User Data

- users/{userId}
: Profile (LMP, Name).
 - appointments/: Sub-collection for calendar.
- userReminderSettings/{userId}_{reminderId}: Reminder configs.
- userTaskStatuses/{userId}_{taskId}_{week}: Task completion status.

TypeScript Types (types/index.ts)

Always refer to types/index.ts when modifying data structures. It contains all interfaces (Week, User, ReminderTemplate, etc.).

6. 📁 Services API

notificationService.ts

- requestNotificationPermissions(): Must be called first.
- scheduleMultipleReminders(id, title, body, hours): Schedules recurring daily notifis.
- cancelReminderNotifications(id): Clears notifis for a specific reminder.

reminderPersistence.ts

Handles the dual-storage strategy (Guest vs Auth).

- saveReminderSettingGuest(...) / saveReminderSettingAuth(...)
- migrateAllGuestDataToAuth(userId): **Critical**. Called on login to move local data to Cloud.

contentService.ts

- getArticlesByIds(ids): Helper to fetch related content.

7. 📁 Scripts & Tooling

Located in app/scripts/. Run with npx ts-node scripts/<name>.ts.

- importV3.ts: Imports Weeks, Reminders, Tasks from JSON to Firestore.
 - importBabyMessages.ts: Imports the 280 baby messages.
 - qaVerification.ts: Runs a health check on Firestore data (counts, links, integrity).
 - debugWeeks.ts: Utility to clean up invalid week documents.
 - generateBabyMessages.ts: Generates the JSON dataset for baby messages.
-

8. 📄 Testing & QA

Unit Tests

Run npm test to execute Jest tests.

- Focus: src/utis/pregnancyCalculator.test.ts (Core logic).

Manual QA Checklist

1. **Migration:** Create guest data -> Login -> Verify data is in Firestore.
 2. **Notifications:** Test on **Physical Device** (Simulators don't handle notifications well).
 3. **Offline:** App should handle offline gracefully (cached data).
-

9. 📄 Deployment

1. **Update Version:** app.json (version, buildNumber).

2. **Build**

:

```
eas build --platform ios
```

```
eas build --platform android
```

3. Submit

:

```
eas submit --platform ios
```

```
eas submit --platform android
```

▮ Notes for Future Developers

- **Translation:** To enable EN/AR content, you must translate the JSON source files and re-run import scripts. The UI is already i18n-ready.
- **Theme:** Use `theme.colors` and `theme.typography` for all styling. Avoid hardcoded hex values.
- **Performance:** HomeScreen loads a lot of data. Keep an eye on `useCurrentWeek` hook performance.

Happy Coding! ☺☺

